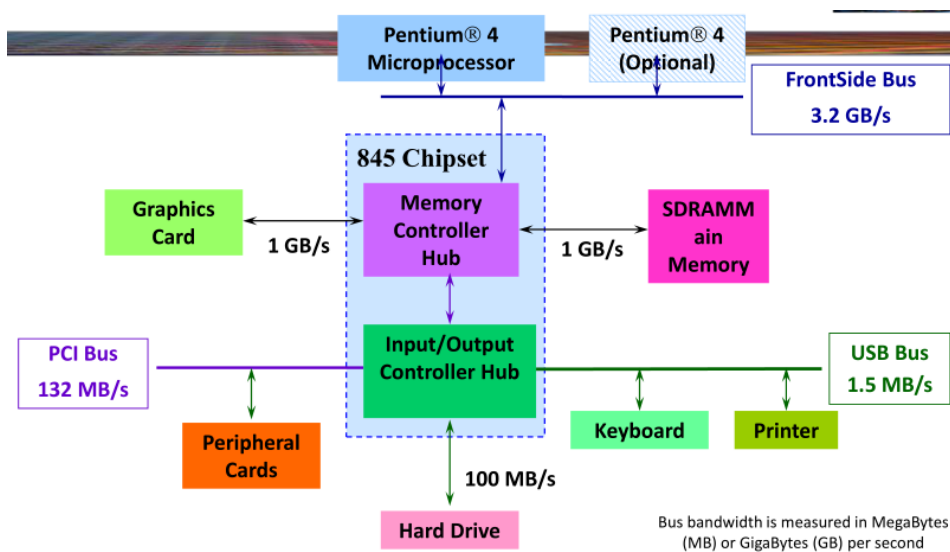


Szybkość procesorów **podwaja się** co **1,8 roku**.

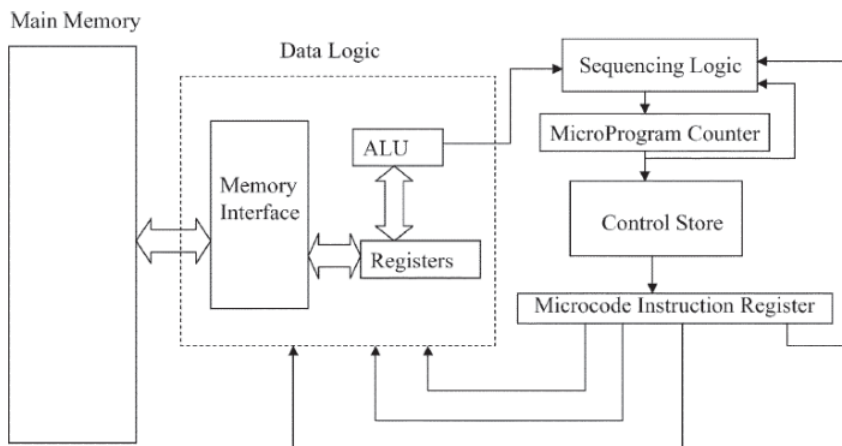
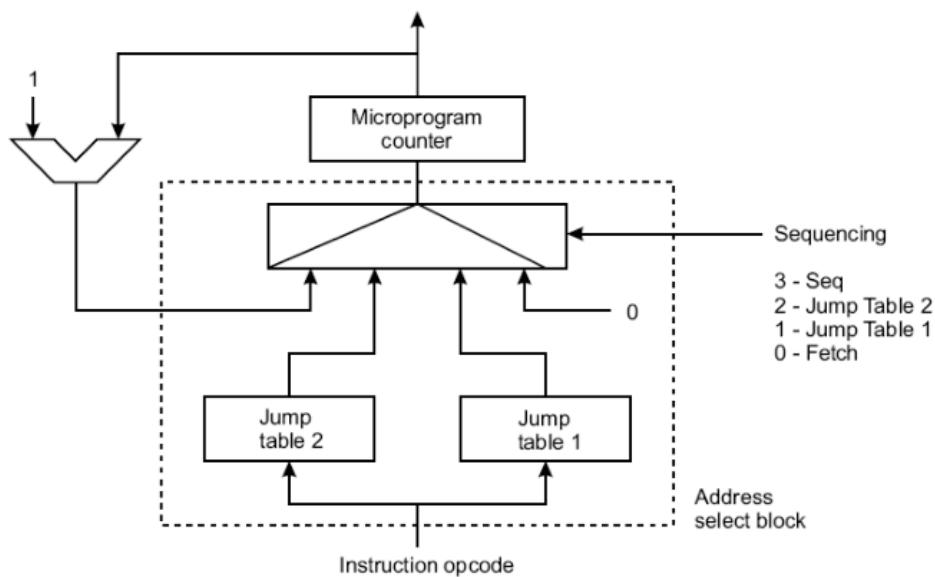
Metoda Tick-Tock – wypuszczanie nowej architektury, a potem w mniejszym procesie technologicznym robienie jej fizycznie mniejszej wersji



mikroprogram – sekwencje mikroinstrukcji realizujące wszystkie instrukcje procesora

Niektóre mikroinstrukcje mogą być wspólne dla instrukcji mikroprocesora, co ogranicza pamięć programu.

Liczba wszystkich mikroinstrukcji może być równa liczbie stanów maszyny stanowej.



ExTime(Y) Performance(X)

----- = ----- = n

ExTime(X) Performance(Y)

Cechy RISC-ów (load/store):

tylko kilka typów instrukcji

brak instrukcji pamięć-pamięć

dane są **ładowane do** rejestrów i **odkładane z nich**

operacje arytmetyczne, logiczne itp. wszystkie są:

- o typu rejestr-rejestr
- o najwyżej mogą mieć **max. 3 operandy: rejestr_docelowy, rejestr_źródłowyA, rejestr_źródłowyB**
- o w ALU większość wykonuje się przez 1 cykl zegara

wszystkie instrukcje są w jednej liczbie bitów (32-bitowe, 64-bitowe), co upraszcza:

- o pre-fetch
- o implementację superskalarności
- o zarządzanie skokami

Prostota instrukcji pozwala na:

duże prędkości zegara

dzielenie instrukcji na mniejsze pod-instrukcje i wykonywanie ich w (super)potoku

Tryby adresowania:

rejestrowe:	add r1, r2
natychmiastowe:	add r1, 10
z przesunięciem:	add r1, 100(r2)
pośrednie rejestrowe:	add r1, (r2)
indeksowe:	add r3, (r1+r2)
bezpośrednie:	add r1, (10)
bezpośrednie z pamięcią:	add r1, @(r2)
z autoinkrementacją:	add r1, (r2)+
z autodekrementacją:	add r1, -(r2)
skalowane:	add r1, 100(r2)[r3]

Najpopularniejsze są:

natychmiastowe:

- o arytmetyka
- o porównania
- o ładowanie stałych

z przesunięciem

datapath – jednostki wykonujące operacje na danych:

IR – rejestr instrukcji

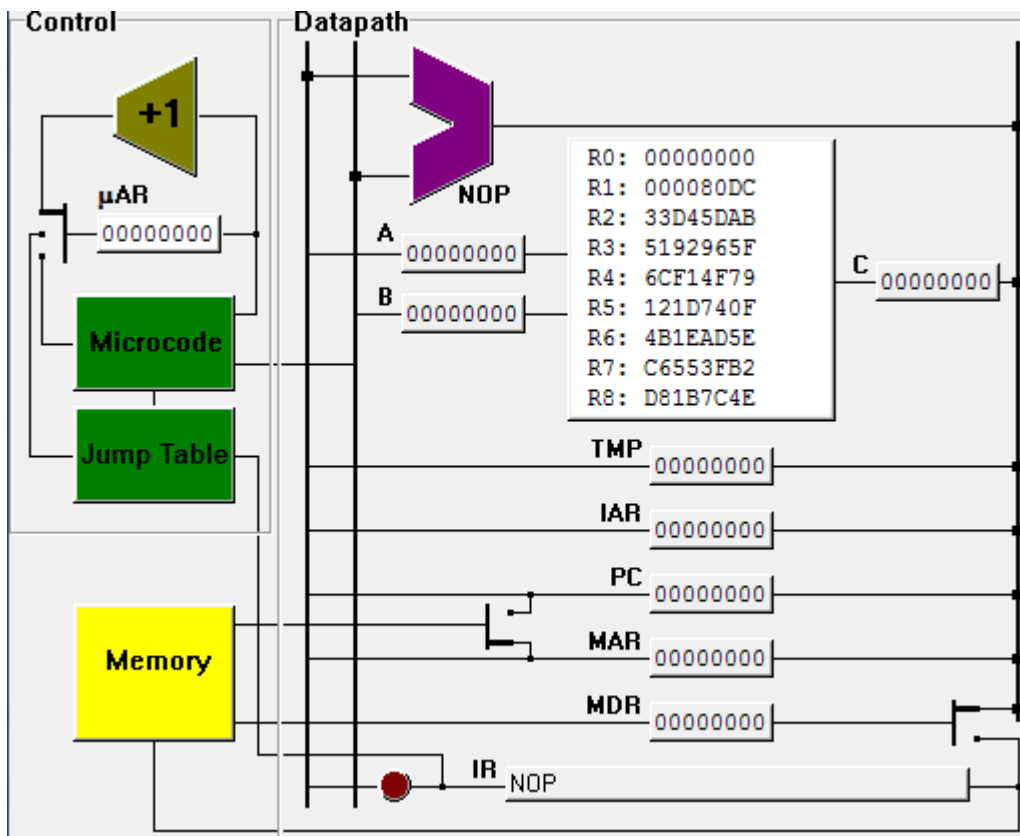
PC – licznik rozkazów

GPR – rejestry ogólnego przeznaczenia

MAR – rejestr adresowy pamięci

MDR – rejestr zawierający dane kierowane do/z pamięci

A, B, C – linie danych



pipelining – pozwala zacząć przetwarzanie kolejnej instrukcji, podczas gdy poprzednia nie jest jeszcze zakończona

Cycle	1	2	3	4	5	6	7	8	9
Instr ₁	Fetch	Decode	Execute	Write					
Instr ₂		Fetch	Decode	Execute	Write				
Instr ₃			Fetch	Decode	Execute	Write			
Instr ₄				Fetch	Decode	Execute	Write		
Instr ₅					Fetch	Decode	Execute	Write	
Instr ₆						Fetch	Decode	Execute	Write

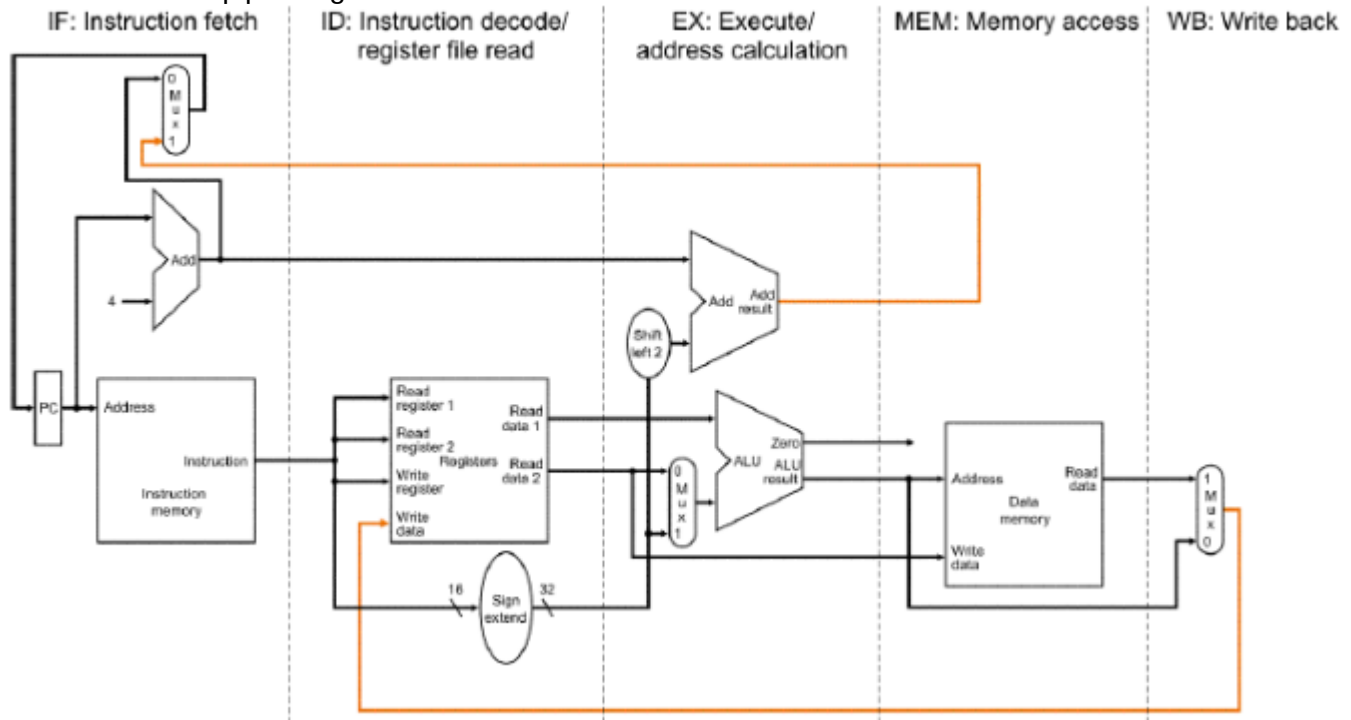
in-order execution – wykonywanie instrukcji bez zmiany ich kolejności – mogą pojawić się przestoje

Cycle	1	2	3	4	5	6	7	8	9
Instr ₁	Fetch	Decode	Execute		Write				
Instr ₂		Fetch	Decode	Wait		Execute	Write		
Instr ₃			Fetch	Decode	Wait		Execute	Write	
Instr ₄				Fetch	Decode	Wait		Execute	Write
Instr ₅					Fetch	Decode	Wait		Execute
Instr ₆						Fetch	Decode	Wait	

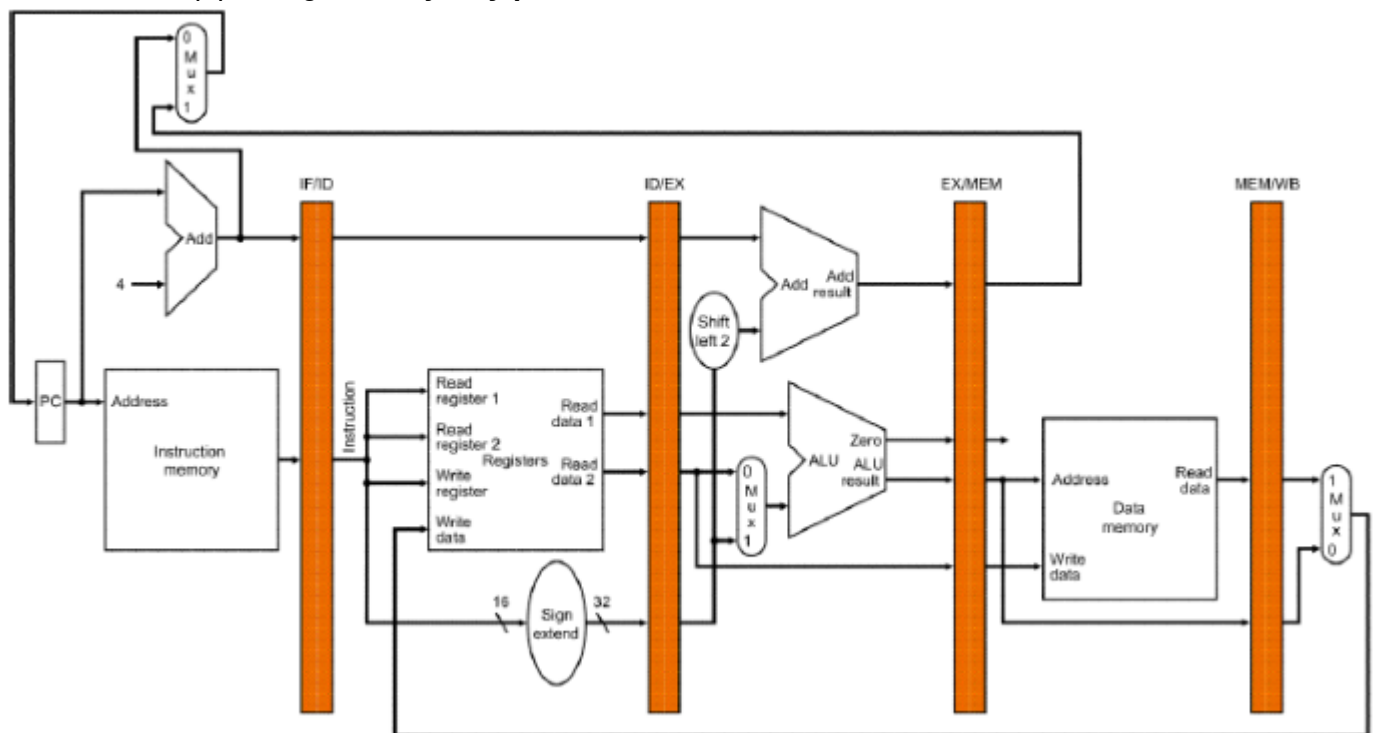
out-of-order execution – jeśli instrukcja wykonywana w ramach jednego potoku ma już dostępne swoje dane wejściowe, to wykonaj od razu

Hyper-Threading – wątek mało obciążony odkłada swoje dane na stos i pomaga wątkowi bardziej obciążonemu

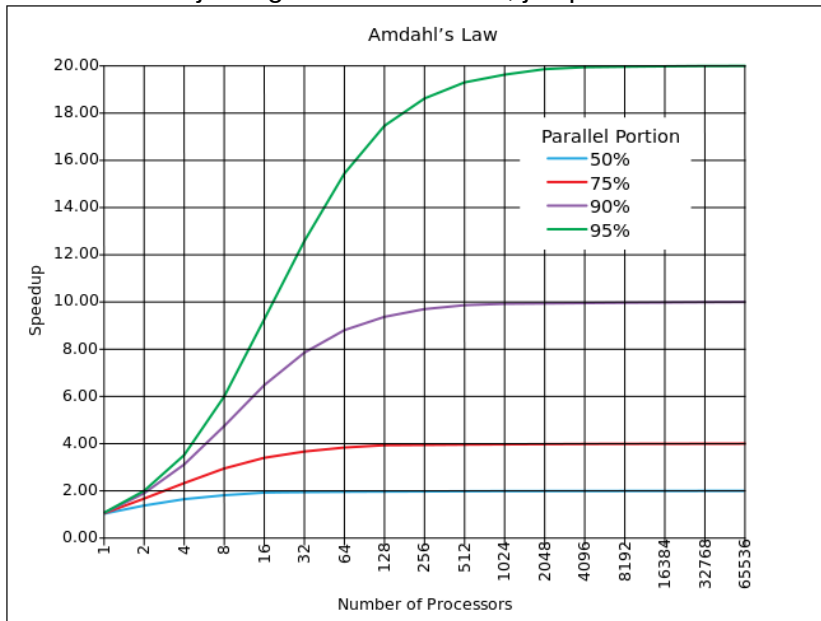
Architektura bez pipeliningu:



Architektura z pipeliningiem – rejestry pośrednie:



programu nie będzie nigdy mniejszy niż ta krytyczna 1 godzina. Tak więc zwiększenie szybkości obliczeń jest ograniczone do 20x, jak przedstawiono na diagramie:



Prawo Gustafsona – każdy wystarczająco duży problem może zostać efektywnie zrównoleglony:

$$S(P) = P - \alpha \cdot (P - 1) \quad \text{gdzie:}$$

S – przyspieszenie

P – ilość procesorów

α – część programu, której nie da się zrównoleglić

delayed branching – najpierw wykonujemy 1-2 instrukcje umieszczone po skoku, a dopiero potem skok

forwarding – jak skończy się wykonywać dana faza potoku, to od razu dostarczamy z niej dane wyjściowe do poprzedniej fazy, która ich potrzebuje

Żelazne prawo:

$$\text{wydajność_procesora} = \text{liczba_instrukcji} \cdot \text{ile_cykli_na_instrukcję} \cdot \text{czas_cyklu}$$

Sposoby mierzenia wydajności:

częstotliwość zegara (nieokładne)

IPC (nie ma sensu bez częstotliwości zegara)

SPEC (Standard Performance Evaluation Corporation) – benchmarki SPECint2K i SPECfp2K – komputery PC

TPC (Transaction Performance Council) – benchmark TPC-C – symulacja typowych obciążeń serwerowych

Ogólnie: architektura procesora definiowana jest przez jego instrukcje.

ARM Cortex:

wysoka wydajność

mały pobór mocy

NEON – operacje typu SIMD

możliwości przetwarzania sygnałów

Jazelle Direct Bytecode Execution – sprzętowo-programowe przetwarzanie kodu bajtowego Javy

TrustZone – zabezpieczenie SoC

Vector Floating Point – koprocessor arytmetyczny

DLX:

uproszczony, bardziej elegancki MIPS, głównie dla celów edukacyjnych
fazy potoku:

- IF
- ID
- EX
- MEM
- WB

VLIW:

ILP (Instruction-Level Parallelism):

- bierzemy kilka instrukcji, np.:
 $f_{12} = f_0 * f_4$, $f_8 = f_8 + f_{12}$, $f_0 = dm(i_0, m_3)$, $f_4 = pm(i_8, m_9)$;
- są pakowane w jeden, długi opkod
- będą wykonane w jednym cyklu zegara (potokowo)

procesory Transmeta i CMS (Code Morphing Software):

- instrukcje x86 są tłumaczone i optymalizowane pod procesor Transmeta
- procesor „uczy się” zachowania programów i optymalizuje wykonanie na bieżąco

Procesory wektorowe:

każdy wynik niezależny od poprzedniego → długi potok, duża częstotliwość zegara
dostęp do pamięci na podstawie znanego wzorca → dobra współpraca z pamięcią, małe opóźnienia, nie ma cache instrukcji

małe problemy ze skokami

jedna instrukcja zabiera dużo pracy → mało pobrań instrukcji (IF)

skalowalne

przewidywalna wydajność w przeciwieństwie do wykorzystania cache, które bazuje na statystyce
dobre do multimediów

Charakterystyki aplikacji:

desktop computing

embedded computing

server computing

